R Environment Setup & Base Graphics

Workshop 1 — Module 8

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. Installing R and RStudio
2. The tidyverse ecosystem
3. R data types and structures for visualization
4. Base R graphics: `plot()`, `hist()`, `barplot()`, `boxplot()`
5. The `par()` system and multi-panel layouts
6. Base R vs ggplot2: when to use each

Learning Outcome
You will have a functioning R environment and be able to produce all four fundamental chart types using base R graphics, preparing the ground for ggplot2 in Workshop 2.

R & RStudio Setup

Installation Checklist

Step 1: Install R

```
# macOS (Homebrew)
brew install r

# Ubuntu / Debian
sudo apt install r-base

# Windows
# Download from https://cran.r-
    project.org
```

Step 2: Install RStudio

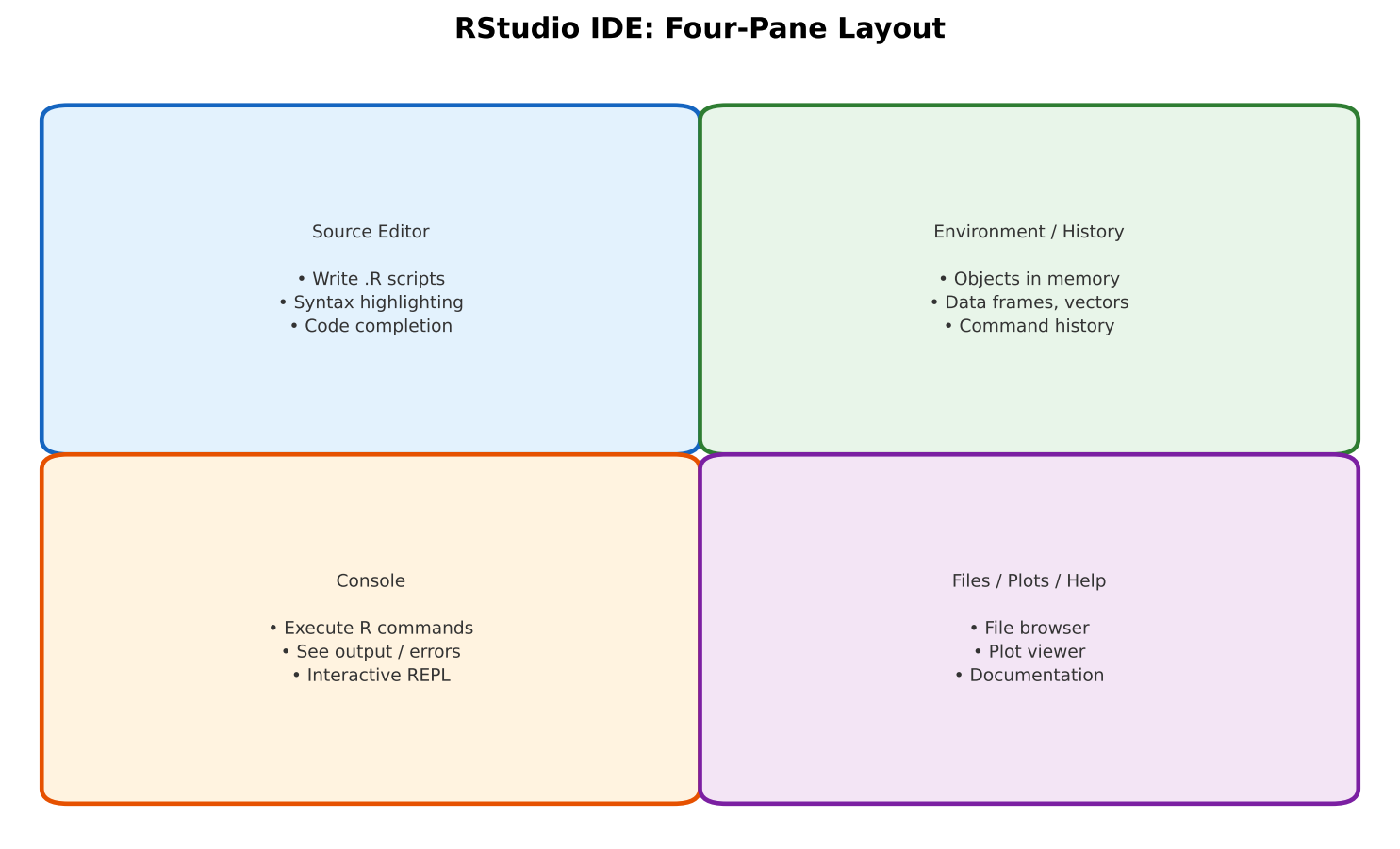
Download from

`posit.co/download/rstudio-desktop`

Step 3: Install core packages

```
# One command installs 8+
    packages
install.packages("tidyverse")

# Additional viz packages
install.packages(c(
  "viridis",
  "RColorBrewer",
  "ggthemes",
  "patchwork",
  "ggrepel",
  "scales"
))
```

The RStudio IDE



RStudio Keyboard Shortcuts

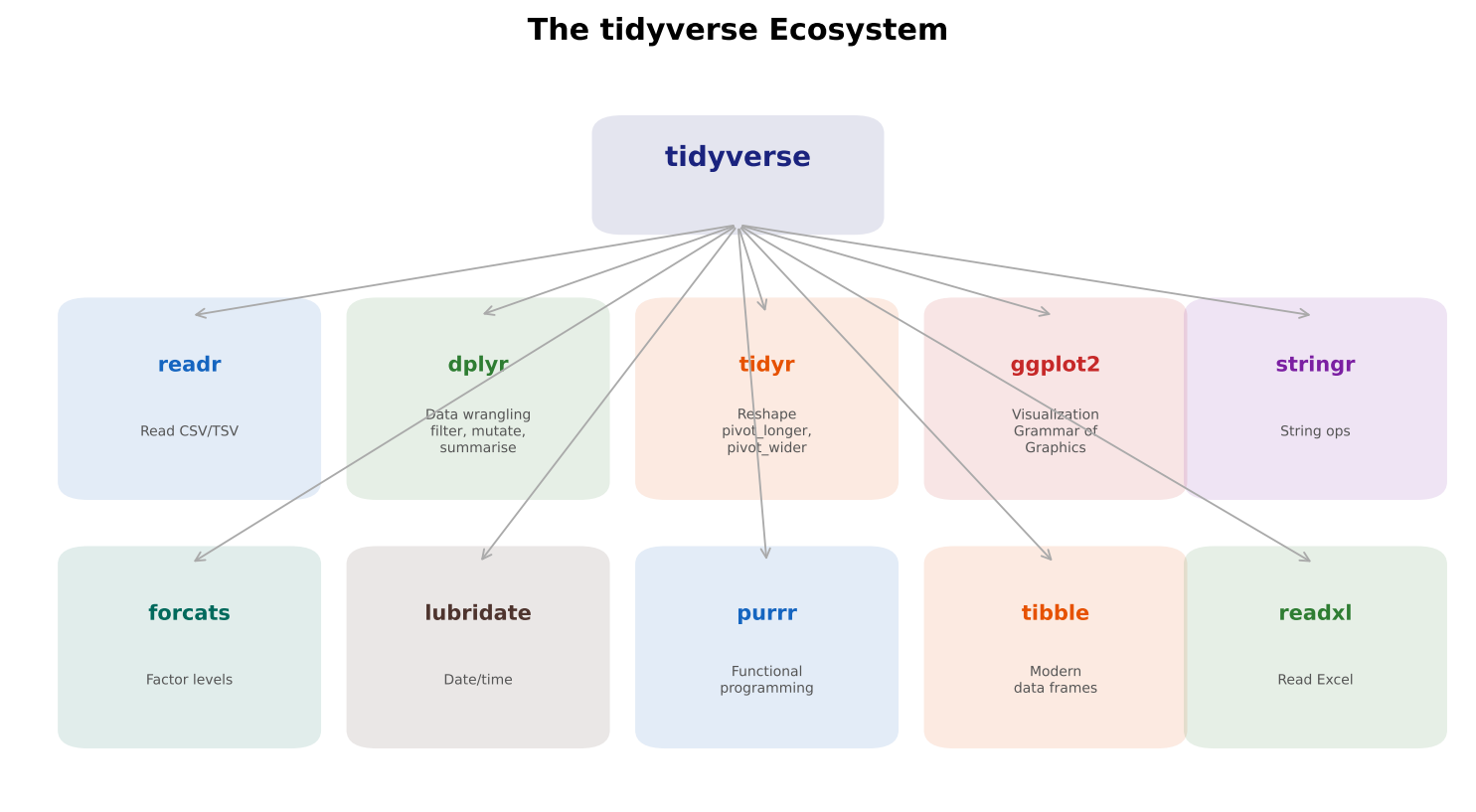
Action	macOS	Windows
Run current line	Cmd+Enter	Ctrl+Enter
Run entire script	Cmd+Shift+Enter	Ctrl+Shift+Enter
Insert pipe |>	Cmd+Shift+M	Ctrl+Shift+M
Insert assignment <-	Option+-	Alt+-
Comment/uncomment	Cmd+Shift+C	Ctrl+Shift+C
New R script	Cmd+Shift+N	Ctrl+Shift+N

Pro-Tip

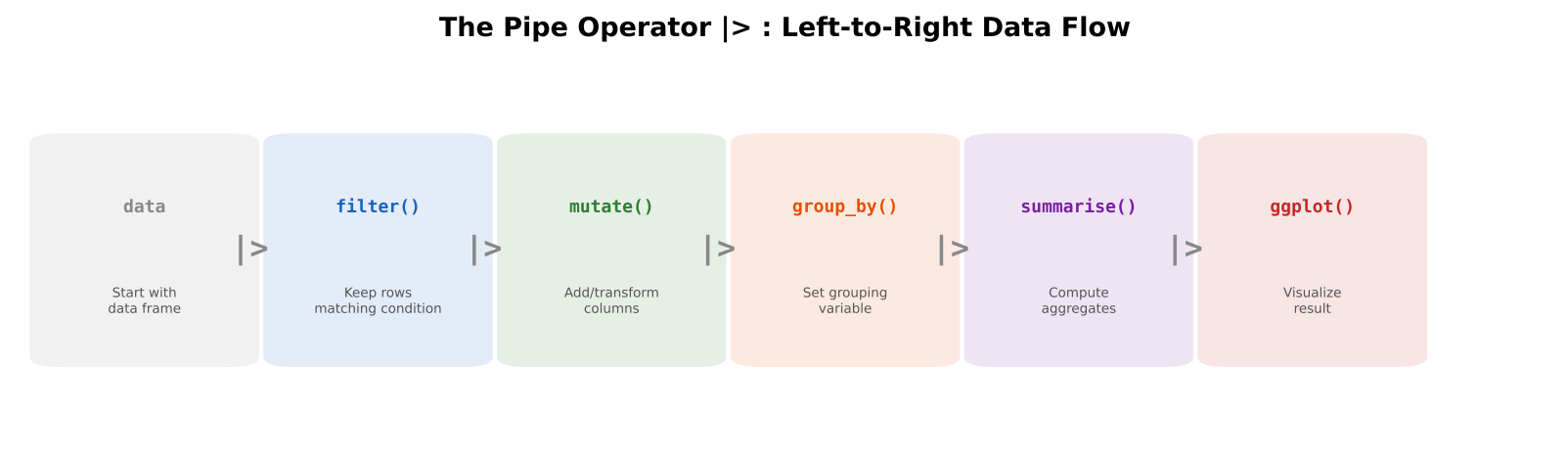
Use `Cmd+Shift+M` / `Ctrl+Shift+M` to insert the pipe operator instantly. This single shortcut will save hours over a term.

The tidyverse Ecosystem

tidyverse: One `library()` Call, Eight Packages



The Pipe: Left-to-Right Data Flow




```r
# Traditional (nested, reads inside-out):
summarise(group_by(filter(data, Type == "Free"), Category), n = n())

# Pipe (linear, reads left-to-right):
data |>
  filter(Type == "Free") |>
  group_by(Category) |>
  summarise(n = n())
```

R Data Types

Data Types for Visualization

R Data Types for Visualization

`numeric`	`42.5, 3.14, -7`	Continuous measurements
`integer`	`1L, 42L, -3L`	Whole numbers (suffix L)
`character`	`"hello", "NYC"`	Text strings
`logical`	`TRUE, FALSE, NA`	Boolean flags
`factor`	`factor(c("A","B"))`	Categorical (ordered/unordered)
`Date`	`as.Date("2024-01-15")`	Calendar dates

Pro-Tip

Use `class(x)` to check a variable's type, and `str(df)` to see all column types in a data frame. Type mismatches (e.g., numbers stored as characters) are the most common source of plotting errors.

Vectors, Data Frames, Tibbles

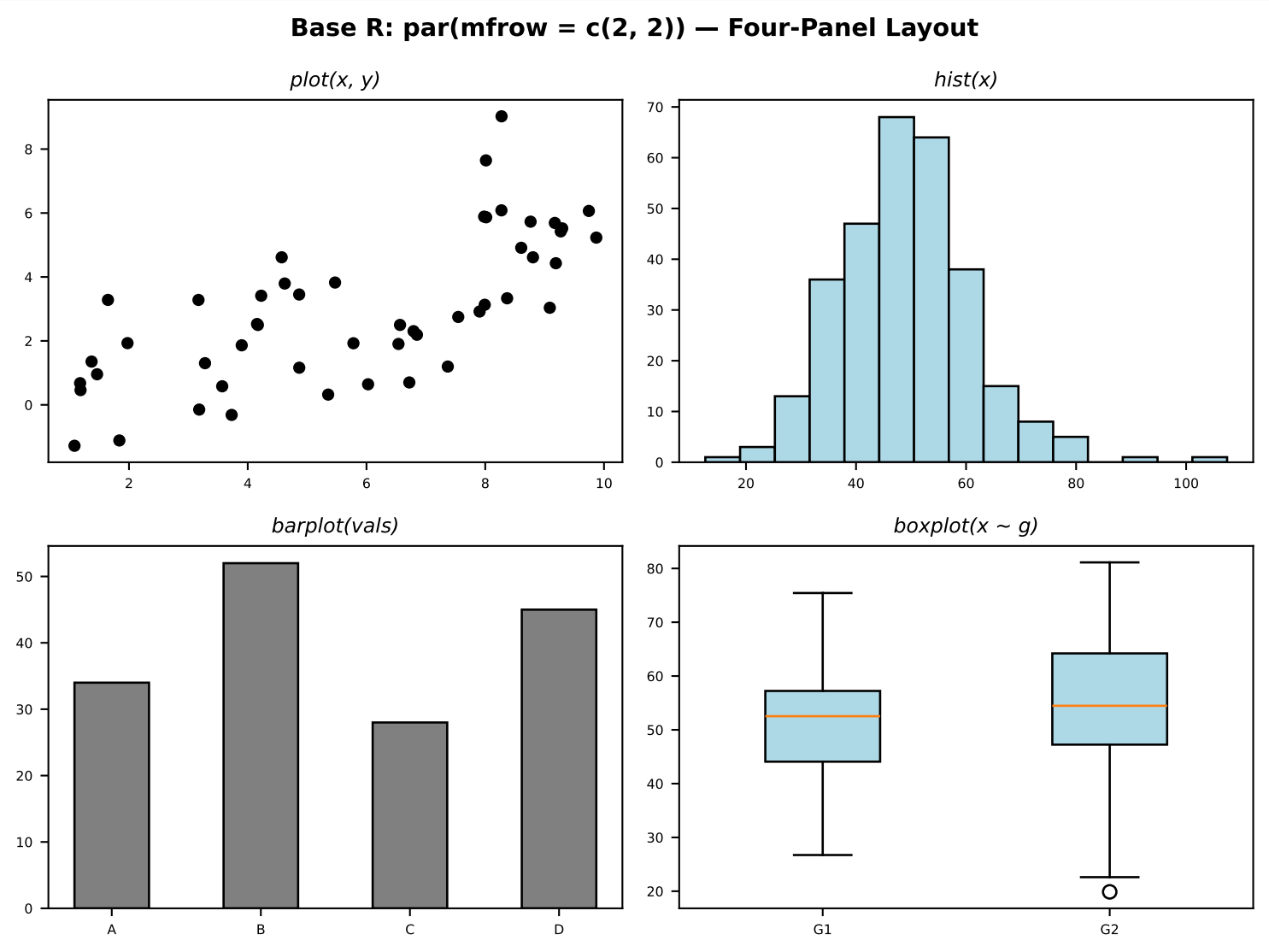
```r
# Vector: 1-dimensional, single type
x <- c(10, 20, 30, 40, 50)
class(x)  # "numeric"

# Data frame: 2D table (base R)
df <- data.frame(
  name = c("Alice", "Bob", "Carol"),
  score = c(88, 92, 75),
  grade = factor(c("A", "A", "B"))
)

# Tibble: modern data frame (tidyverse)
library(tibble)
tb <- tibble(
  name  = c("Alice", "Bob", "Carol"),
  score = c(88, 92, 75),
  grade = factor(c("A", "A", "B"))
)

# Key difference: tibbles print compactly
# and never convert strings to factors
print(tb)
```

Loading and Inspecting Data

```r
library(readr)

# Read CSV (tidyverse     fast, type-guessing)
apps <- read_csv("googleplaystore.csv")

# Quick inspection
dim(apps)          # rows x columns
glimpse(apps)      # compact column view
head(apps, 5)      # first 5 rows
summary(apps)      # five-number summaries

# Check for problems
problems(apps)     # parsing issues
sum(is.na(apps)) # total missing values

# Useful for viz planning:
# - How many rows? (affects point density)
# - Column types? (determines chart choice)
# - Missing values? (need filtering/imputation)
```

Base R Graphics

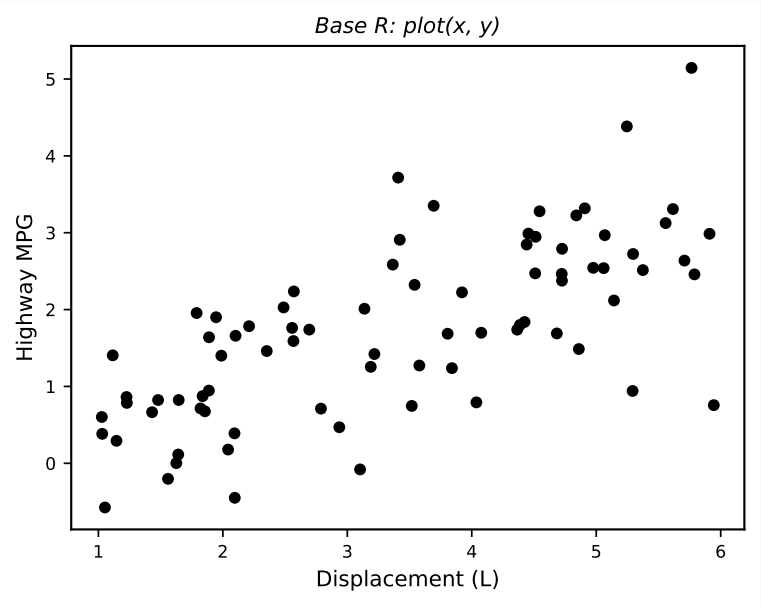
The Four Fundamental Chart Types



Data Insight
Base R produces functional charts instantly with zero package

Scatterplot: `plot(x, y)`

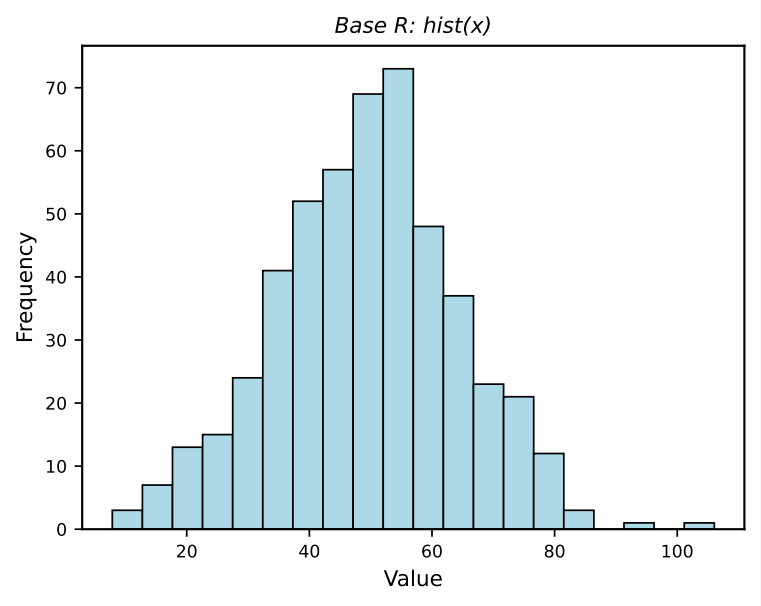
```r
# Minimal call
plot(mpg$displ, mpg$hwy)

# With customisation
plot(mpg$displ, mpg$hwy,
  main = "Engine vs Fuel Economy",
  xlab = "Displacement (L)",
  ylab = "Highway MPG",
  pch  = 16,        # solid circle
  col  = "steelblue",
  cex  = 0.8)       # point size

# Add regression line
abline(
  lm(hwy ~ displ, data = mpg),
  col = "red", lwd = 2)

# Add legend
legend("topright",
  legend = "OLS fit",
  col = "red", lwd = 2)
```



Key parameters: `pch` = point shape (16 = filled), `col` = colour, `cex` = size multiplier, `lwd` = line width.

Histogram: `hist(x)`

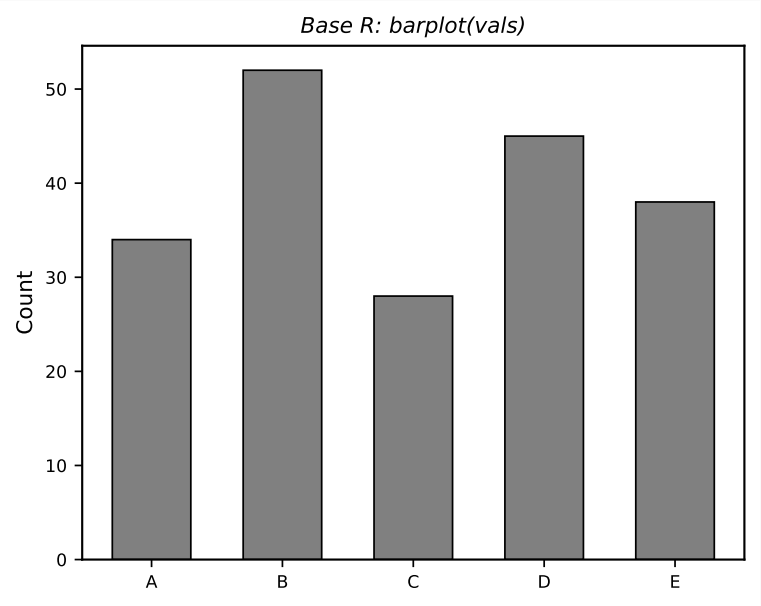
```r
# Minimal call
hist(apps$Rating)

# With customisation
hist(apps$Rating,
  breaks = 30,            # bin count
  col    = "lightblue",
  border = "white",
  main   = "App Rating Distribution
       ",
  xlab   = "Rating (1-5)",
  ylab   = "Frequency",
  las    = 1)             #
       horizontal labels

# Add density curve overlay
lines(density(apps$Rating,
  na.rm = TRUE),
  col = "red", lwd = 2)

# Add vertical mean line
abline(v = mean(apps$Rating,
  na.rm = TRUE),
  col = "blue", lwd = 2,
  lty = 2)  # dashed
```



Key parameters: `breaks` = number of bins, `border` = bar edge colour, `las = 1` = horizontal tick labels.

Bar Chart: `barplot()`

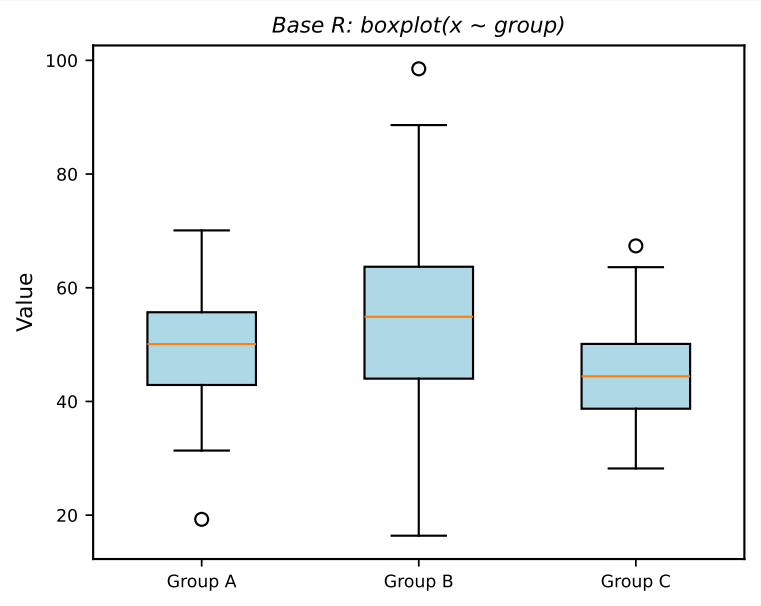
```r
# Count categories
counts <- table(apps$Type)

# Minimal bar chart
barplot(counts)

# Customised horizontal bar
barplot(sort(counts),
  horiz    = TRUE,
  col      = "steelblue",
  border   = NA,
  main     = "Apps by Type",
  xlab     = "Count",
  las      = 1,
  cex.names = 0.8)

# Grouped bar chart
barplot(
  matrix(c(30, 50, 20, 40), nrow =
      2),
  beside = TRUE,
  col = c("steelblue", "coral"),
  names.arg = c("Cat A", "Cat B"),
  legend.text = c("Free", "Paid"))
```



Key parameters: `horiz = TRUE` flips to horizontal, `beside = TRUE` groups bars side by side (vs. stacked), `border = NA` removes bar edges.

Box Plot: `boxplot()`

```r
# Formula syntax: y ~ group
boxplot(Rating ~ Type, data = apps,
  main   = "Rating by App Type",
  ylab   = "Rating",
  col    = c("lightblue", "salmon")
     ,
  border = "grey40",
  notch  = TRUE,    # CI for median
  las    = 1)

# Add mean points
means <- tapply(apps$Rating,
  apps$Type, mean, na.rm = TRUE)
points(1:2, means,
  pch = 18, col = "red", cex = 1.5)

# Horizontal boxplot
boxplot(Rating ~ Type, data = apps,
  horizontal = TRUE)
```



The formula `y ~ group` is R's standard interface for conditioning: "y split by group." Used in boxplot, lm, aov, etc.

The par() System

Multi-Panel Layouts with `par(mfrow)`

```r
# Set up 2x2 grid (fills row by row)
par(mfrow = c(2, 2))

# Panel 1: scatter
plot(mpg$displ, mpg$hwy, pch = 16, cex = 0.6,
     main = "Scatter", xlab = "Displacement", ylab = "MPG")

# Panel 2: histogram
hist(mpg$hwy, breaks = 20, col = "lightblue", border = "white",
     main = "Distribution", xlab = "Highway MPG")

# Panel 3: bar chart
barplot(table(mpg$class), col = "steelblue", border = NA,
        main = "Vehicle Class", las = 2, cex.names = 0.7)

# Panel 4: boxplot
boxplot(hwy ~ cyl, data = mpg, col = "lightblue",
        main = "MPG by Cylinders", xlab = "Cylinders", ylab = "MPG")

# Reset to single plot
par(mfrow = c(1, 1))
```

Customising `par()` Parameters

Parameter	Effect
`mfrow = c(r, c)`	Grid with r rows, c columns
`mar = c(b, l, t, r)`	Margins in lines (bottom, left, top, right)
`mgp = c(3, 1, 0)`	Axis title, labels, line positions
`cex.main = 1.2`	Title size multiplier
`cex.axis = 0.8`	Tick label size multiplier
`bg = "white"`	Background colour
`las = 1`	Horizontal tick labels

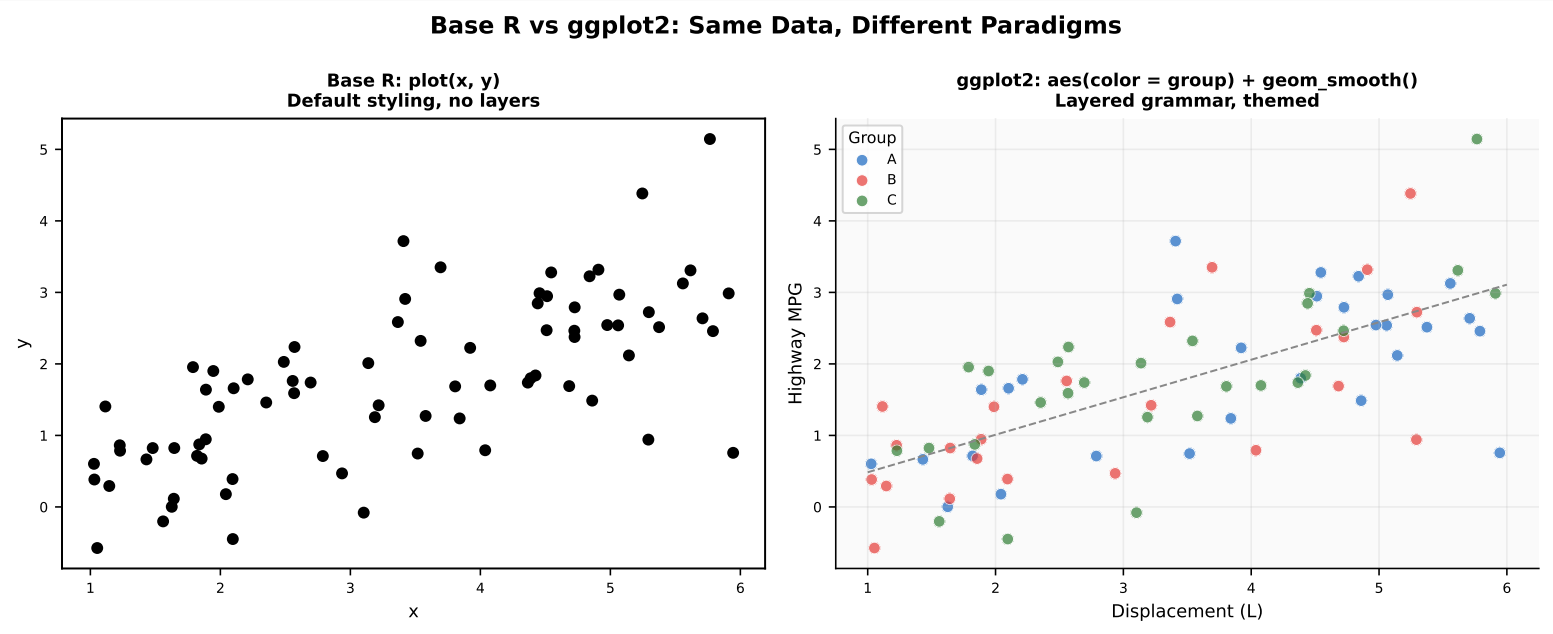
Pro-Tip

Always save and restore `par()`:

```
old_par <- par(mfrow = c(2,2))
# ...  plots ...
par(old_par)
```

Base R vs ggplot2

When to Use Each



Base R vs ggplot2: Decision Guide

Criterion	Base R	ggplot2
Speed to first plot	Fastest (1 line)	Slightly more setup
Customisation	Manual, verbose	Declarative, layered
Faceting	`par(mfrow)`	`facet_wrap()`
Themes	Manual `par()`	`theme_*()` library
Colour scales	Manual vectors	`scale_*` functions
Publication quality	Requires effort	Near-ready by default
Learning curve	Shallow start	Steeper, but compounds

Data Insight

Use **base R** for quick exploratory checks ("what does this variable look like?"). Use **ggplot2** for anything that will be seen by others. We cover ggplot2 in depth in Workshop 2.

Synthesis

Key Takeaways

1. **RStudio** is the standard IDE: learn the four panes and the pipe shortcut (`Cmd/Ctrl+Shift+M`).
2. **tidyverse** loads eight essential packages with one `library()` call. Master `readr` + `dplyr` + `ggplot2` first.
3. **Data types** determine chart choices: check with `class()` and `str()` before plotting.
4. **Base R graphics** (`plot`, `hist`, `barplot`, `boxplot`) produce instant exploratory charts with zero dependencies.
5. **par(mfrow)** creates multi-panel layouts; always save and restore `par()` settings.
6. Base R is for **exploration**; ggplot2 (Workshop 2) is for **explanation and publication**.

Essential Readings for This Module

- Wickham, H. & Grolemund, G. (2023). *R for Data Science*, 2nd ed., Chapters 1–3 (Setup, Workflow, Data Transformation). `https://r4ds.hadley.nz`
- Murrell, P. (2018). *R Graphics*, 3rd ed., Chapters 1–3 (Base Graphics System).
- Peng, R. D. (2016). *Exploratory Data Analysis with R*, Chapters 4–6 (Plotting Systems).

Next Module: Python Environment Setup & Matplotlib Basics (W01-M09)